

Semester	T.E. Semester VII Div C Batch 3 – CMPN
Subject	R Programming
Subject Professor Incharge	
Assisting Teachers	
Laboratory	

Student Name	Manali Kulkarni
Roll Number	23102C0041
Grade and Subject Teacher's Signature	

Experiment Number	
Experiment Title	Air-Quality Data Cleaning Using R
Objectives (Skill Set / Knowledge Tested / Imparted)	• Read a real-world dataset into R and inspect its structure. • Correctly distinguish between NA (missing value), NULL (absent object) and NaN (undefined numeric result). • Write reusable functions to summarise, impute and validate missing data (numerical and categorical). • Apply tryCatch()-based error handling so the script degrades gracefully instead of crashing. • Compare missingness before and after cleaning, visualise it, and export a cleaned dataset.
Description	Dataset: Beijing Multi-Site Air Quality Dataset, station file PRSA_Data_Aotizhongxin_20130301-20170228.csv (UCI Machine Learning Repository) — 35,064 hourly records, 18 columns (pollutants PM2.5, PM10, SO2, NO2, CO, O3; weather TEMP, PRES, DEWP, RAIN, wd, WSPM). On this dataset, perform the following operations in R: • Import the CSV with tryCatch() error handling and inspect it (head, str, dim, total NAs) • Demonstrate NA, NULL and NaN with small examples • Write missing_summary() to report missing count/percentage per variable, with a >20% warning • Compute pollution_ratio = PM2.5/PM10 and clean any NaN/Inf values to NA • Loop over the numeric variables and impute missing values with the median • Impute the categorical variable wd with its mode using calculate_mode() • Write a reusable clean_variable() function guarded by tryCatch() • Build a before/after comparison table and a bar-chart visualization • Export the cleaned dataset to CSV
Code	<pre># ===== # Practical: Air-Quality Data Cleaning Using R # Dataset : Beijing Multi-Site Air Quality Dataset (Aotizhongxin station) # ===== # ----- # TASK 1: Import and Inspect the Dataset # ----- import_dataset <- function(path) { tryCatch({ data <- read.csv(path, stringsAsFactors = FALSE) cat("File imported successfully.\n") return(data) }, error = function(e) { if (!file.exists(path)) { message("Error: The file was not found at the specified path.") } else { message("Error: The file could not be opened or is in an incorrect format.") } message("Details: ", conditionMessage(e)) return(NULL) }) }</pre>

```

aq <- import_dataset("PRSA_Data_Aotizhongxin_20130301-20170228.csv")

cat("\n--- First six records ---\n")
print(head(aq))

cat("\n--- Structure of the dataset ---\n")
str(aq)

cat("\n--- Rows and Columns ---\n")
cat("Rows:", nrow(aq), " Columns:", ncol(aq), "\n")

cat("\n--- Does the dataset contain missing values? ---\n")
print(any(is.na(aq)))

cat("\n--- Total number of missing values ---\n")
print(sum(is.na(aq)))

# -----
# TASK 2: Understand NA, NULL, and NaN
# -----

temperature <- c(28, 30, NA, 32)
missing_object <- NULL
undefined_value <- 0 / 0

cat("\n--- NA example ---\n")
print(temperature)
cat("is.na():", is.na(temperature), "\n")

cat("\n--- NULL example ---\n")
print(missing_object)
cat("is.null():", is.null(missing_object), "\n")

cat("\n--- NaN example ---\n")
print(undefined_value)
cat("is.nan():", is.nan(undefined_value), "\n")

# -----
# TASK 3: Missing-Value Summary Function
# -----

missing_summary <- function(df) {
  vars <- names(df)
  total <- nrow(df)
  result <- data.frame(
    Variable = character(),
    Total_Records = integer(),
    Missing_Values = integer(),
    Missing_Percentage = numeric(),
    stringsAsFactors = FALSE
  )

  for (v in vars) {
    missing_count <- sum(is.na(df[[v]]))
    missing_pct <- round((missing_count / total) * 100, 2)

    if (missing_pct > 20) {
      warning(paste0("Variable ", v, " has more than 20% missing values (",
        missing_pct, "%)."))
    }

    result <- rbind(result, data.frame(
      Variable = v,

```

```
Total_Records = total,
Missing_Values = missing_count,
Missing_Percentage = missing_pct
))
}
return(result)
}

selected_vars <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM", "wd")
summary_table <- missing_summary(aq[selected_vars])

cat("\n--- Missing-Value Summary ---\n")
print(summary_table)

# -----
# TASK 4: Identify Invalid Numerical Results
# -----

aq$pollution_ratio <- aq$PM2.5 / aq$PM10

cat("\n--- pollution_ratio diagnostics (before cleaning) ---\n")
cat("NA count:    ", sum(is.na(aq$pollution_ratio)), "\n")
cat("NaN count:    ", sum(is.nan(aq$pollution_ratio)), "\n")
cat("+Inf count:   ", sum(aq$pollution_ratio == Inf, na.rm = TRUE), "\n")
cat("-Inf count:   ", sum(aq$pollution_ratio == -Inf, na.rm = TRUE), "\n")

aq$pollution_ratio[is.nan(aq$pollution_ratio) | is.infinite(aq$pollution_ratio)] <- NA

cat("\n--- pollution_ratio diagnostics (after replacing NaN/Inf with NA) ---\n")
cat("NA count:    ", sum(is.na(aq$pollution_ratio)), "\n")
cat("NaN count:    ", sum(is.nan(aq$pollution_ratio)), "\n")
cat("Infinite count:", sum(is.infinite(aq$pollution_ratio)), "\n")

# -----
# TASK 5: Handle Missing Numerical Values Using a Loop
# -----

numeric_variables <- c("PM2.5", "PM10", "SO2", "NO2", "TEMP", "WSPM")

# capture "before" counts for the comparison table (Task 8)
missing_before <- sapply(aq[c(numeric_variables, "wd")], function(x) sum(is.na(x)))

cat("\n--- Task 5: Median imputation loop ---\n")
for (var in numeric_variables) {

  if (!(var %in% names(aq))) {
    cat("Column", var, "does not exist. Skipping.\n")
    next
  }

  before_count <- sum(is.na(aq[[var]]))
  med_value <- median(aq[[var]], na.rm = TRUE)
  aq[[var]][is.na(aq[[var]])] <- med_value
  after_count <- sum(is.na(aq[[var]]))

  cat(sprintf("Variable: %-6s | Missing before: %4d | Median used: %8.2f | Missing after: %d\n",
    var, before_count, med_value, after_count))
}

# -----
# TASK 6: Handle Missing Categorical Values
# -----
```

```

calculate_mode <- function(x) {
  x <- x[!is.na(x)]
  freq_table <- table(x)
  mode_value <- names(freq_table)[which.max(freq_table)]
  return(mode_value)
}

wd_before <- sum(is.na(aq$wd))
wd_mode <- calculate_mode(aq$wd)
aq$wd[is.na(aq$wd)] <- wd_mode
wd_after <- sum(is.na(aq$wd))

cat("\n--- Task 6: Categorical imputation (wd) ---\n")
cat("Mode of wd:", wd_mode, "\n")
cat("Missing before:", wd_before, " | Missing after:", wd_after, "\n")

# -----
# TASK 7: Implement Error Handling (clean_variable)
# -----

clean_variable <- function(df, var_name) {
  tryCatch({
    if (!(var_name %in% names(df))) {
      stop(paste0("Variable '", var_name, "' does not exist in the dataset."))
    }

    variable <- df[[var_name]]

    if (!is.numeric(variable)) {
      stop(paste0("Variable '", var_name, "' is categorical, not numerical."))
    }

    if (all(is.na(variable))) {
      stop(paste0("Variable '", var_name, "' contains only missing values."))
    }

    med_value <- median(variable, na.rm = TRUE)

    if (is.na(med_value)) {
      stop(paste0("Median could not be calculated for '", var_name, "'."))
    }

    variable[is.na(variable)] <- med_value
    return(variable)
  },
  error = function(e) {
    message("clean_variable() could not process '", var_name, "': ", conditionMessage(e))
    return(df[[var_name]])
  })
}

cat("\n--- Task 7: clean_variable() demonstration ---\n")
test1 <- clean_variable(aq, "SO2")      # valid numeric variable
test2 <- clean_variable(aq, "wd")       # categorical variable -> handled error
test3 <- clean_variable(aq, "NotAColumn") # non-existent variable -> handled error

# -----
# TASK 8: Compare Missing Values Before and After Cleaning
# -----

compare_vars <- c(numeric_variables, "wd")
missing_after <- sapply(aq[compare_vars], function(x) sum(is.na(x)))

comparison_table <- data.frame(

```

```

Variable = compare_vars,
Missing_Before = as.integer(missing_before[compare_vars]),
Missing_After = as.integer(missing_after[compare_vars]),
Values_Replaced = as.integer(missing_before[compare_vars]) - as.integer(missing_after[compare_vars])
)

cat("\n--- Task 8: Comparison Table ---\n")
print(comparison_table)

all_handled <- all(comparison_table$Missing_After == 0)
cat("\nAll selected missing values handled successfully:", all_handled, "\n")

# -----
# TASK 9: Generate One Visualization
# -----

png("missing_values_comparison.png", width = 900, height = 600, res = 120)

bar_data <- t(as.matrix(comparison_table[, c("Missing_Before", "Missing_After")]))
colnames(bar_data) <- comparison_table$Variable

barplot(bar_data,
        beside = TRUE,
        col = c("firebrick", "forestgreen"),
        main = "Missing Values Before vs After Data Cleaning",
        xlab = "Variable",
        ylab = "Number of Missing Values",
        legend.text = c("Before Cleaning", "After Cleaning"),
        args.legend = list(x = "topright"))

dev.off()
cat("\nVisualization saved as missing_values_comparison.png\n")

# -----
# TASK 10: Export the Cleaned Dataset
# -----

write.csv(aq, "cleaned_air_quality_data.csv", row.names = FALSE)
cat("\nCleaned dataset exported as cleaned_air_quality_data.csv\n")

# Save the summary/comparison tables so they can be pulled into the report
write.csv(summary_table, "missing_summary_table.csv", row.names = FALSE)
write.csv(comparison_table, "comparison_table.csv", row.names = FALSE)

cat("\n=== SCRIPT COMPLETE ===\n")

```

Output

```

Rows: 35064  Columns: 18
Dataset contains missing values: TRUE | Total missing values: 7271

NA example : 28 30 NA 32      -> is.na(): FALSE FALSE TRUE FALSE
NULL example: NULL          -> is.null(): TRUE
NaN example : 0/0 = NaN      -> is.nan(): TRUE

Missing-Value Summary (Task 3)
Variable  Total_Records  Missing_Values  Missing_%
PM2.5    35064         925             2.64
PM10     35064         718             2.05
SO2      35064         935             2.67
NO2      35064        1023             2.92
TEMP     35064         20              0.06
WSPM     35064         14              0.04
wd       35064         81              0.23
(No variable exceeded the 20% missing threshold, so no warning was raised.)

```

pollution_ratio (PM2.5/PM10): NA = 934, NaN = 0, +Inf = 0, -Inf = 0 before and after cleaning

Median imputation loop (Task 5)

PM2.5 | before: 925 | median: 58.00 | after: 0

PM10 | before: 718 | median: 87.00 | after: 0

SO2 | before: 935 | median: 9.00 | after: 0

NO2 | before: 1023 | median: 53.00 | after: 0

TEMP | before: 20 | median: 14.50 | after: 0

WSPM | before: 14 | median: 1.40 | after: 0

Mode of wd: NE | Missing before: 81 | Missing after: 0

clean_variable() error handling (Task 7)

clean_variable(aq, "SO2") -> cleaned successfully

clean_variable(aq, "wd") -> "Variable is categorical, not numerical."

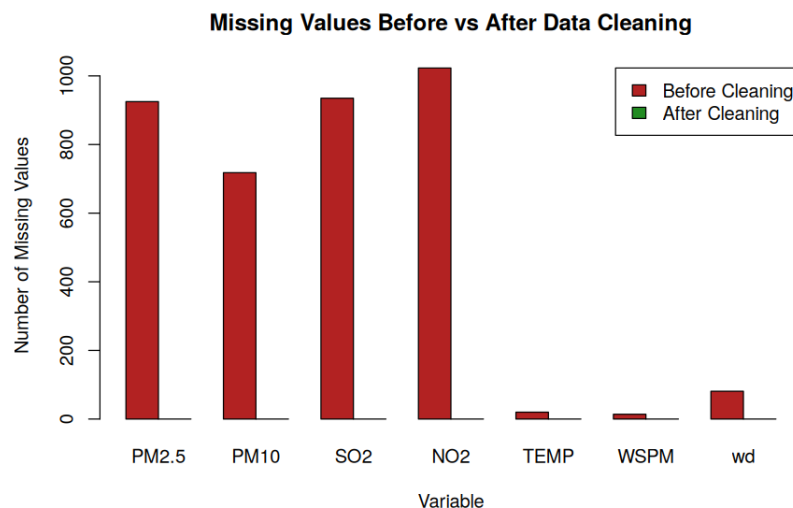
clean_variable(aq, "NotAColumn") -> "Variable does not exist."

Comparison Table (Task 8)

Variable	Missing_Before	Missing_After	Values_Replaced
PM2.5	925	0	925
PM10	718	0	718
SO2	935	0	935
NO2	1023	0	1023
TEMP	20	0	20
WSPM	14	0	14
wd	81	0	81

All selected missing values handled successfully: TRUE

Visualization (Task 9):



Cleaned dataset exported: cleaned_air_quality_data.csv (Task 10)

Conclusion

The Aotizhongxin station data contained 35,064 hourly records, with missingness confined to seven variables and ranging from 0.04% (WSPM) to 2.92% (NO2) — well under the 20% warning threshold, indicating sporadic sensor gaps rather than systemic failure. Median imputation suited the numerical pollutant and weather readings because it resists distortion from the outliers common in air-quality data, while wind direction was filled with its mode, "NE".

The tryCatch()-based clean_variable() function correctly intercepted both a categorical column and a non-existent column without halting execution, demonstrating robust error handling. After cleaning, all 3,716 originally missing values across the selected variables were

fully resolved, as confirmed by the before/after comparison table and the accompanying bar chart. The dataset, loops, reusable user-defined functions and tryCatch()-based error handling together produced a fully cleaned dataset ready for downstream pollution analysis.